Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»

Номер зачетной книжки _25100021__
Производственная практика зачтена с оценкой
_____ (_____)
(цифрой) (прописью)
_____
(подпись руководителя практики от БГУИР)
____._____.2026

ОТЧЕТ
по преддипломной практике
Место прохождения практики: РУП «Центр цифрового развития», г.Минск
Сроки прохождения практики: с 9.02.2026 по 22.03.2026

Руководитель практики от предприятия:
_____ Д.М.Омельчук
(подпись руководителя)

Студент группы 251001
_____ Д.В.Гатальский
(подпись студента)
Руководитель практики от БГУИР
Хмелев А.Г. – д.э.н., профессор кафедры ПОИТ

Минск 2026

ВВЕДЕНИЕ

В условиях активного развития цифровых технологий и финансовых сервисов возрастает необходимость эффективного управления личными финансами. Увеличение количества финансовых операций и использование различных источников доходов и расходов усложняют контроль за денежными средствами. В результате пользователи часто сталкиваются с проблемами неэффективного распределения бюджета и отсутствия инструментов для анализа расходов.

Целью преддипломной практики является подготовка и обоснование основных решений дипломного проекта, направленного на повышение эффективности управления личными финансами пользователя за счёт разработки системы анализа и оптимизации персональных расходов.

В рамках преддипломной практики особое внимание уделяется формированию первых трёх разделов дипломного проекта. Для достижения поставленной цели необходимо решить следующие задачи:

1) провести анализ информационных источников по теме дипломного проекта (существующие системы, методы, алгоритмы и программные решения);

2) разработать задание на дипломное проектирование;

3) разработать техническое задание на создание программного средства;

4) провести анализ требований к проектируемому программному средству и сформировать функциональные требования;

5) выполнить проектирование архитектуры программного средства;

6) разработать обобщённый алгоритм функционирования системы и алгоритмы отдельных модулей;

Объектом исследования является процесс управления личными финансами пользователя.

Предметом исследования являются методы и программные средства анализа и оптимизации персональных расходов.

В первом разделе выполняется анализ предметной области и формирование требований. Во втором разделе рассматривается проектирование системы. В третьем разделе описывается реализация программного средства.

1 Анализ прототипов, литературных источников и формирование требований к проектируемому программному средству

1.1 Анализ предметной области

В последние годы наблюдается устойчивый рост интереса к цифровым инструментам управления личными финансами. Распространение мобильных устройств, онлайн-банкинга и финансовых технологий способствует активному развитию систем, предназначенных для анализа расходов, планирования бюджета и повышения финансовой грамотности пользователей. Данные системы позволяют автоматизировать учет финансовых операций, анализировать структуру расходов, формировать финансовые цели и принимать более обоснованные решения при управлении личным бюджетом.

Дополнительно следует отметить, что развитие цифровой экономики привело к увеличению объема финансовых данных, генерируемых пользователями. Это обусловило необходимость создания интеллектуальных систем, способных не только фиксировать финансовые операции, но и осуществлять их комплексный анализ. В частности, современные программные средства все чаще включают функции прогнозирования расходов, выявления аномалий в финансовом поведении и формирования рекомендаций по оптимизации бюджета [4].

Актуальность разработки таких систем подтверждается динамикой развития рынка. По данным аналитических исследований, мировой рынок приложений для управления личными финансами в 2023 году оценивался примерно в 101,75 млрд долларов США, а к 2032 году может достичь 675 млрд долларов США при среднем ежегодном темпе роста около 23,4 % [1]. По оценкам аналитиков, количество пользователей подобных приложений может достигнуть 1,8 млрд человек к 2025 году, что свидетельствует о значительном росте популярности цифровых инструментов для контроля расходов и планирования бюджета [2].

С точки зрения информационных технологий, системы управления личными финансами относятся к классу информационно-аналитических систем, предназначенных для сбора, хранения, обработки и визуализации данных. В таких системах широко применяются методы анализа данных, включая статистические методы, алгоритмы классификации и прогнозирования, а также инструменты визуальной аналитики. Использование подобных методов позволяет пользователям получать наглядное представление о своих финансовых потоках и выявлять неэффективные модели расходования средств [5].

Современные системы управления личными финансами предоставляют пользователям функции автоматической категоризации расходов, построения аналитических отчетов и визуализации финансовых данных. Исследования показывают, что около 60 % пользователей при выборе финансового приложения обращают внимание прежде всего на функции бюджетирования и анализа расходов [3]. При этом важную роль играет удобство пользовательского интерфейса и скорость взаимодействия с системой, что напрямую влияет на уровень вовлеченности пользователей [6].

Особое значение в данной предметной области имеет обеспечение безопасности и конфиденциальности данных. Финансовые приложения работают с чувствительной информацией, включая сведения о доходах, расходах и банковских операциях пользователей. В связи с этим необходимо применение современных методов защиты информации, таких как шифрование данных, многофакторная аутентификация и разграничение прав доступа [7].

Таким образом, разработка информационных систем для анализа и оптимизации персональных расходов является актуальной задачей, направленной на повышение эффективности управления личными финансами, улучшение финансовой дисциплины пользователей и внедрение интеллектуальных методов анализа данных в повседневную практику.Анализ существующих аналогов

На современном рынке представлено большое количество программных решений, предназначенных для управления личными финансами и анализа расходов. Такие системы позволяют пользователям отслеживать доходы и расходы, формировать бюджеты, анализировать структуру затрат и получать рекомендации по оптимизации финансов. Развитие данного класса программных средств обусловлено ростом цифровых финансовых сервисов, доступностью мобильных устройств и необходимостью повышения финансовой грамотности населения.

Современные приложения для управления личными финансами реализуют широкий спектр функций, включая автоматическую синхронизацию с банковскими счетами, интеллектуальную категоризацию транзакций, формирование аналитических отчетов, а также визуализацию финансовых данных в виде графиков и диаграмм. Дополнительно некоторые системы используют методы анализа данных для прогнозирования расходов и формирования персонализированных рекомендаций пользователю [4].

Подобные приложения можно условно классифицировать по нескольким категориям:

1) Приложения для учета личных финансов (например, Mint, Wallet, Monefy) — ориентированы на фиксацию доходов и расходов, категоризацию

транзакций и формирование базовых отчетов. Основной акцент в таких системах делается на простоте использования и скорости ввода данных;

2) Системы бюджетирования и финансового планирования (например, YNAB — You Need a Budget, PocketGuard) — предоставляют инструменты для планирования бюджета, постановки финансовых целей, контроля лимитов расходов и анализа отклонений от запланированных показателей. Такие системы часто используют определенные методики финансового управления;

3) Мобильные финансовые менеджеры (например, CoinKeeper, Zenmoney) — предназначены для повседневного контроля расходов и удобного ввода транзакций с мобильных устройств. Они ориентированы на оперативность и доступность, обеспечивая пользователю быстрый доступ к информации о текущем состоянии бюджета;

4) Веб-ориентированные финансовые платформы — представляют собой комплексные решения, обеспечивающие доступ к системе через браузер и интеграцию с различными сервисами. Такие системы чаще всего обладают расширенными аналитическими возможностями и поддерживают работу с большими объемами данных.

Согласно аналитическим исследованиям, более 60 % пользователей финансовых приложений используют их прежде всего для контроля расходов и анализа бюджета, что подтверждает высокую востребованность подобных систем [3]. При этом рынок приложений для управления личными финансами продолжает активно расти благодаря распространению мобильных технологий, развитию открытых банковских API и увеличению числа пользователей цифровых финансовых сервисов [2].

Несмотря на широкий выбор существующих решений, многие из них имеют ряд ограничений, связанных с зависимостью от банковских интеграций, ограниченной доступностью в отдельных регионах, наличием платных функций, а также недостаточно развитой аналитикой и возможностями прогнозирования. Это определяет необходимость разработки новых систем, направленных на устранение указанных недостатков и повышение эффективности управления личными финансами пользователей [5].

1.1.1 Mint

Mint — одно из наиболее известных и широко распространенных приложений для управления личными финансами, разработанное компанией Intuit. Данный сервис представляет собой веб-ориентированную платформу, которая позволяет пользователям централизованно управлять своими финансовыми данными за счет интеграции с банковскими счетами, кредитными картами и другими финансовыми инструментами.

Основной особенностью Mint является высокий уровень автоматизации процессов учета и анализа финансов. Система осуществляет автоматическую загрузку транзакций, их обработку и последующую категоризацию, что позволяет значительно снизить участие пользователя в процессе ведения учета. Кроме того, Mint предоставляет инструменты для формирования бюджетов, анализа расходов и визуализации финансовой информации.

Достоинства Mint:

1) Автоматическая синхронизация с банковскими счетами. Приложение автоматически импортирует транзакции из банковских аккаунтов и кредитных карт, что значительно упрощает процесс учета финансов;

2) Автоматическая категоризация расходов. Система анализирует транзакции и распределяет их по категориям (питание, транспорт, развлечения и др.), позволяя пользователю быстро анализировать структуру расходов;

3) Функции бюджетирования. Пользователь может устанавливать лимиты расходов для различных категорий и получать уведомления при их превышении;

4) Наглядная визуализация данных. Приложение предоставляет графики и диаграммы, позволяющие анализировать финансовую активность;

5) Централизация финансовой информации. Mint объединяет данные из различных источников, что обеспечивает комплексный подход к управлению личными финансами.

Недостатки Mint:

1) Ограниченная доступность. Сервис ориентирован преимущественно на пользователей США и Канады, что делает его использование затруднительным в других странах;

2) Зависимость от банковской интеграции. Без подключения банковских счетов функциональность приложения значительно снижается;

3) Вопросы конфиденциальности. Некоторые пользователи выражают обеспокоенность необходимостью предоставления доступа к банковским данным;

4) Ограниченные возможности настройки. Пользовательские сценарии работы с системой имеют определенные ограничения, что может снижать гибкость использования приложения;

5) Отсутствие продвинутого прогнозирования. Несмотря на развитые функции учета и анализа, Mint не предоставляет расширенных инструментов прогнозирования финансового поведения пользователя.

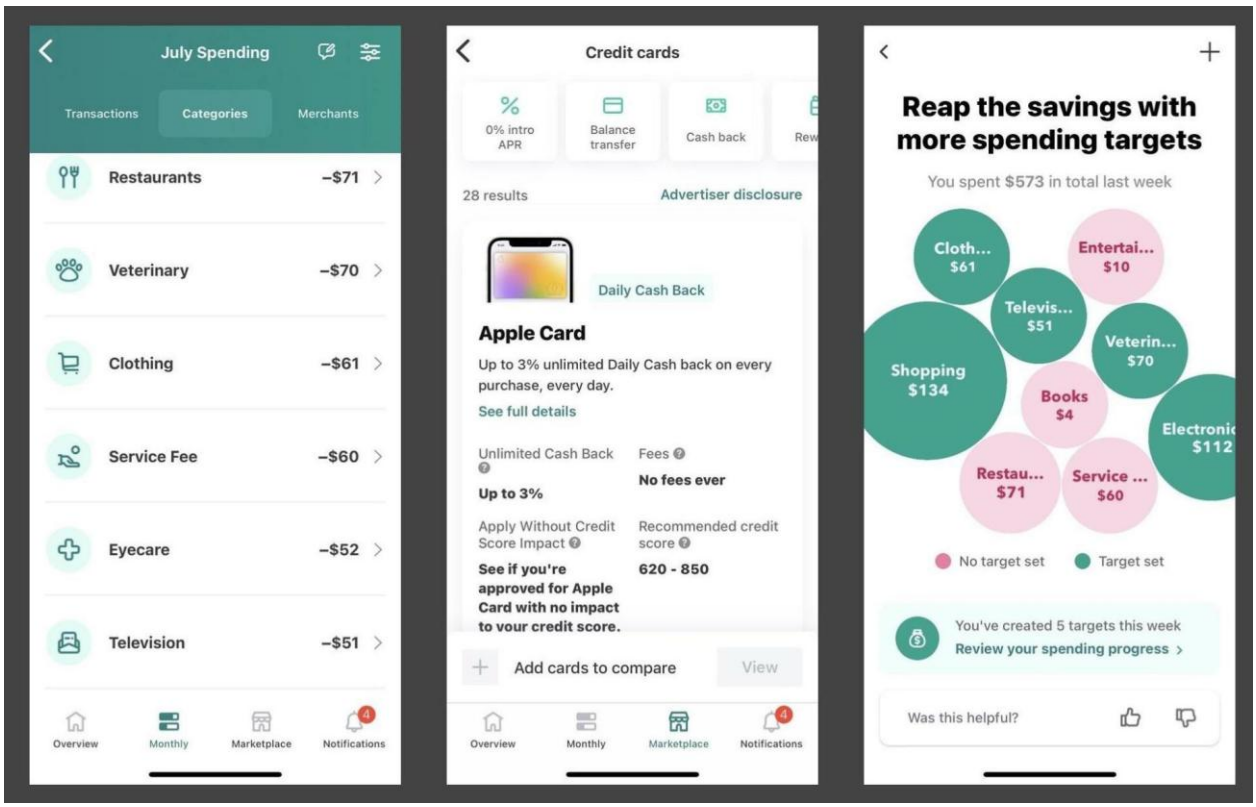

Рисунок 1.1 – Интерфейс Mint

В целом, Mint является мощным инструментом для автоматизированного учета и анализа личных финансов, однако его использование ограничено географическими факторами, зависимостью от внешних интеграций и вопросами безопасности. Проведенный анализ показывает, что при разработке нового программного средства целесообразно учитывать сильные стороны данного решения, такие как автоматизация и визуализация, одновременно устраняя выявленные недостатки.

1.1.2 YNAB

YNAB — популярное приложение для планирования личного бюджета, основанное на концепции проактивного распределения доходов. В основе системы лежит принцип, согласно которому каждый полученный доход должен быть заранее распределен по категориям расходов, что позволяет пользователю более осознанно подходить к управлению своими финансами.

В отличие от многих других систем, ориентированных преимущественно на учет уже совершенных операций, YNAB делает акцент на планировании будущих расходов и формировании устойчивых финансовых привычек. Методология, используемая в системе, базируется на четырех ключевых правилах: распределение каждого дохода, контроль текущих расходов, адаптация бюджета к изменениям и формирование финансового

резерва. Такой подход способствует повышению финансовой дисциплины пользователей и снижению вероятности возникновения дефицита бюджета

Достоинства YNAB:

1) Продвинутая система бюджетирования. Приложение помогает пользователю планировать расходы заранее и контролировать финансовые цели;

2) Методика финансового планирования. YNAB использует собственную систему финансовой дисциплины, направленную на формирование устойчивых финансовых привычек;

3) Кроссплатформенность. Сервис доступен как в виде веб-приложения, так и в мобильных версиях для различных операционных систем;

4) Образовательные материалы. Платформа предоставляет обучающие ресурсы и курсы по финансовому планированию.

Недостатки YNAB:

1) Платная подписка. Для полноценного использования требуется ежемесячная или годовая подписка;

2) Сложность освоения. Пользователям может потребоваться время для понимания методики бюджетирования, используемой в системе;

3) Ограниченные аналитические возможности. Несмотря на развитую систему бюджетирования, инструменты анализа финансовых данных представлены достаточно ограниченно;

4) зависимость от интернет-соединения.

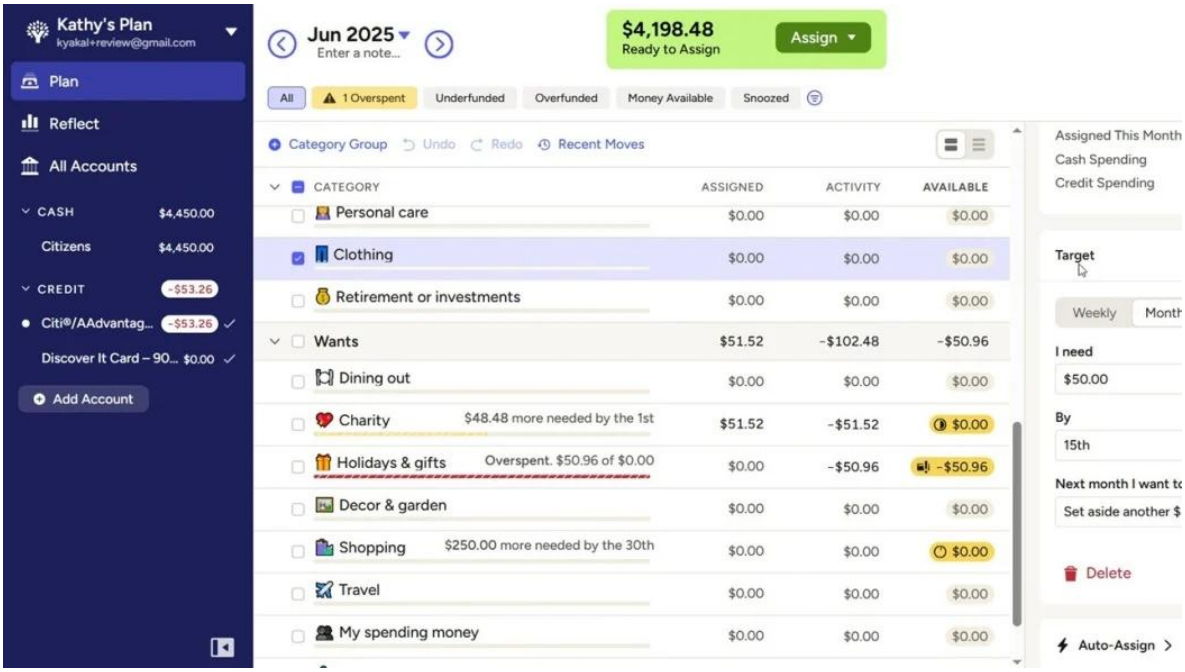

Рисунок 1.2 – Интерфейс YNAB

В целом, YNAB представляет собой эффективный инструмент для планирования бюджета и формирования финансовой дисциплины пользователей. Однако акцент на методологии бюджетирования и ограниченные аналитические возможности определяют необходимость разработки систем, которые сочетают удобство использования, автоматизацию учета и расширенные инструменты анализа финансовых данных.

1.1.3 Zenmoney

Zenmoney — популярное приложение для управления личными финансами, широко используемое в русскоязычных странах. Данное программное средство ориентировано на комплексный учет доходов и расходов, анализ финансовой активности пользователей, а также интеграцию с банковскими сервисами для автоматизации обработки финансовых данных.

Система Zenmoney сочетает в себе функции учета, анализа и планирования бюджета, обеспечивая пользователю возможность получения полной картины финансового состояния. Приложение поддерживает автоматическую загрузку транзакций, их категоризацию и последующую визуализацию, что позволяет эффективно отслеживать динамику расходов и доходов.

Особенностью Zenmoney является ориентация на пользователей стран СНГ, что выражается в поддержке локальных банковских сервисов, валют и специфики финансовых операций. Это делает приложение более доступным и удобным по сравнению с зарубежными аналогами.

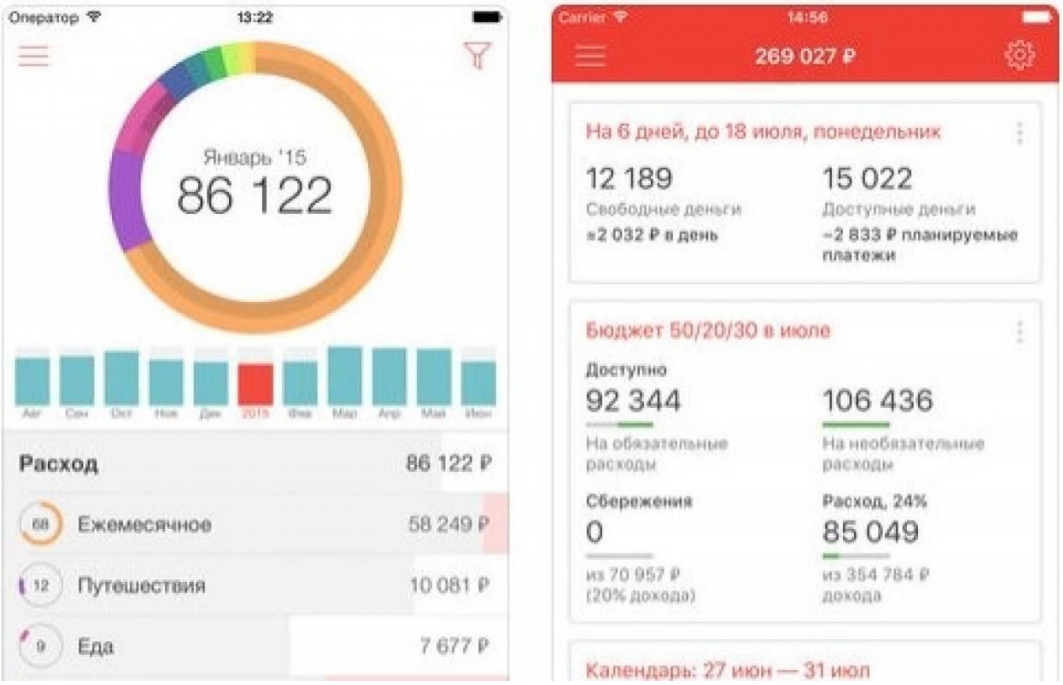

Рисунок 1.3 – Интерфейс приложения Zenmoney

Преимущества Zenmoney:

1) Поддержка банковских интеграций. Приложение может автоматически импортировать транзакции из различных банковских сервисов;

2) Подробная аналитика расходов. Пользователь может анализировать структуру затрат по категориям, периодам времени и источникам доходов;

3) Кроссплатформенность. Система доступна на мобильных устройствах и через веб-интерфейс;

4) Поддержка различных валют. Приложение удобно использовать для учета финансов при международных транзакциях.

Недостатки Zenmoney:

1) Ограниченный функционал бесплатной версии. Некоторые функции доступны только при покупке подписки;

2) Зависимость от банковских интеграций. При отсутствии поддержки банка часть функций становится недоступной;

3) Ограниченные возможности прогнозирования расходов.

В целом, Zenmoney является эффективным инструментом для учета и анализа личных финансов, особенно для пользователей русскоязычного сегмента. Однако наличие ограничений, связанных с платным функционалом и недостаточной глубиной прогнозной аналитики, указывает на необходимость разработки систем, сочетающих доступность, независимость от внешних сервисов и расширенные аналитические возможности.

1.1.4 Monefy

Monefy — популярное мобильное приложение для учета личных финансов, ориентированное на быстрый и удобный ввод данных о доходах и расходах. В отличие от более сложных систем, таких как Mint или YNAB, Monefy делает акцент на простоте использования и минимизации действий пользователя при работе с приложением.

Основной принцип работы системы заключается в ручном вводе финансовых операций с последующей их категоризацией и визуализацией. Приложение не требует обязательной интеграции с банковскими счетами, что делает его более универсальным и независимым от внешних сервисов.

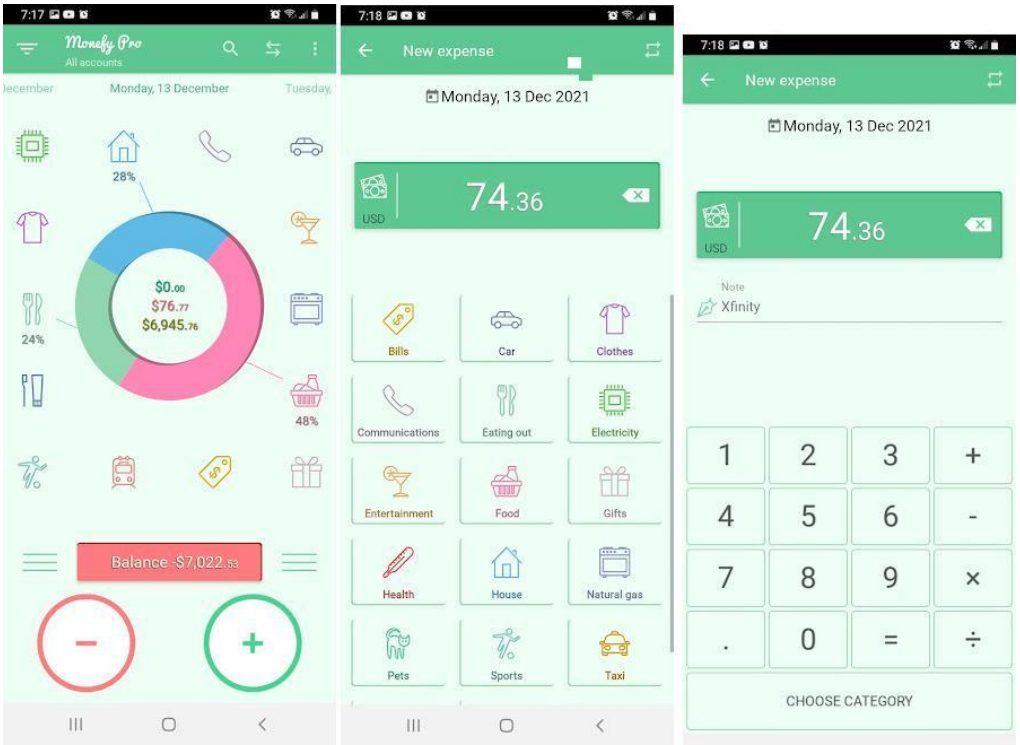

Рисунок 1.4 – Интерфейс приложения Monefy

Преимущества Monefy:

1) Простота использования. Интерфейс приложения интуитивно понятен и позволяет быстро добавлять операции, что особенно удобно для повседневного использования;

2) Независимость от банковских интеграций. Все данные вводятся пользователем вручную, что позволяет использовать приложение без подключения к банковским сервисам;

3) Высокая скорость работы. Минималистичный дизайн и отсутствие сложных вычислений обеспечивают высокую производительность приложения;

4) Наглядная визуализация. Приложение предоставляет диаграммы и графики, позволяющие быстро оценить структуру расходов;

5) Поддержка мультивалютности. Пользователь может учитывать операции в различных валютах;

6) Подробная аналитика расходов. Пользователь может анализировать структуру затрат по категориям, периодам времени и источникам доходов.

Недостатки Monefy:

1) Отсутствие автоматизации. Все операции необходимо вводить вручную, что увеличивает трудоемкость учета и может приводить к ошибкам;

2) Ограниченные аналитические возможности. Приложение предоставляет базовую визуализацию, но не поддерживает глубокий анализ и прогнозирование расходов;

3) Отсутствие интеграции с банками. Невозможность автоматического получения данных снижает удобство использования по сравнению с более продвинутыми системами;

4) Ограниченный функционал планирования. Возможности бюджетирования и постановки финансовых целей реализованы на базовом уровне;

5) Ограничения бесплатной версии. Некоторые функции доступны только в платной версии приложения.

В целом, Monefy является удобным инструментом для базового учета личных финансов, ориентированным на пользователей, предпочитающих простоту и контроль над своими данными. Однако отсутствие автоматизации и развитых аналитических возможностей ограничивает его применение в задачах комплексного анализа и оптимизации персональных расходов.

1.1.5 PocketGuard

PocketGuard — популярное приложение для управления личными финансами, ориентированное на анализ расходов и контроль бюджета пользователя. Основная концепция сервиса заключается в предоставлении пользователю информации о сумме денежных средств, доступных для безопасного расходования после учета обязательных платежей, регулярных расходов и финансовых целей. Такой подход позволяет минимизировать риск перерасхода и способствует более рациональному управлению личными финансами.

Система PocketGuard автоматически обрабатывает финансовые операции, полученные через банковские интеграции, и формирует сводную информацию о текущем состоянии бюджета. Особенностью приложения является ориентация на упрощение восприятия финансовых данных: пользователю предоставляется краткая и наглядная информация без необходимости глубокого анализа.

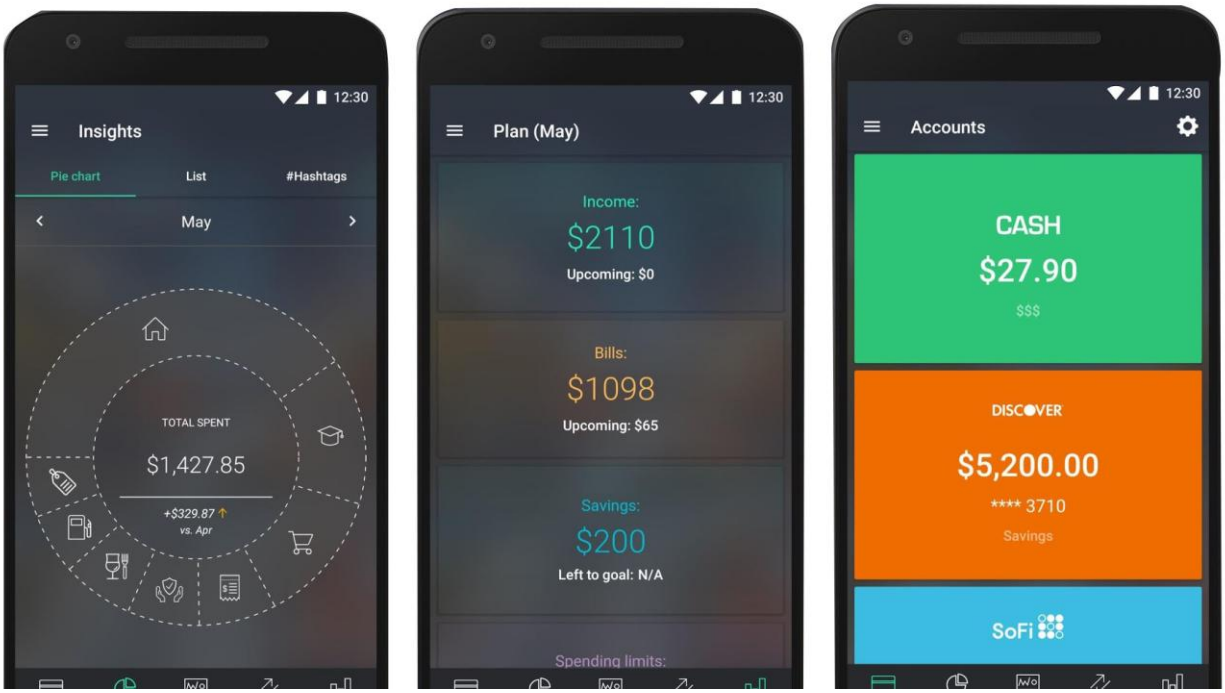

Рисунок 1.5 – Интерфейс приложения PocketGuard

Преимущества PocketGuard:

1) Функция “In My Pocket”. Приложение автоматически рассчитывает сумму денег, которая остаётся у пользователя после учета регулярных платежей, счетов и финансовых целей, что помогает избежать перерасхода бюджета;

2) Автоматическая категоризация расходов. Система анализирует транзакции и распределяет их по категориям, что облегчает анализ структуры расходов;

3) Интеграция с банковскими счетами. PocketGuard может синхронизироваться с банковскими счетами и кредитными картами, автоматически импортируя транзакции;

4) Аналитика расходов. Пользователь может просматривать статистику расходов по категориям, периодам времени и источникам дохода.

Недостатки PocketGuard:

1) Ограниченная доступность банковских интеграций. Большинство банковских подключений ориентированы на пользователей США и Канады;

2) Ограниченный функционал бесплатной версии. Расширенные функции бюджетирования и аналитики доступны только в платной версии PocketGuard Plus;

3) Недостаточно развитые функции финансового прогнозирования. Приложение в основном анализирует текущие расходы, но не предоставляет продвинутых инструментов долгосрочного планирования бюджета;

В целом, PocketGuard является удобным инструментом для оперативного контроля расходов и предотвращения перерасхода бюджета. Однако его функциональные ограничения, связанные с недостаточной глубиной аналитики и зависимостью от банковских интеграций, подтверждают необходимость разработки более универсальных систем, сочетающих простоту использования с расширенными аналитическими возможностями.

1.2 Требования к проектируемому программному средств

1.2.1 Назначение разработки

Проектируемое программное средство предназначено для автоматизации учета персональных финансов пользователей, включая регистрацию доходов и расходов, их категоризацию, а также аналитическую обработку финансовых данных. Приложение должно предоставлять пользователям удобный инструментарий для контроля личного бюджета, отслеживания динамики расходов и повышения эффективности управления финансовыми ресурсами

1.2.2 А Состав выполняемых функций

Разрабатываемое веб-приложение должно обеспечивать выполнение следующего основного функционала:

1) автоматизированный учет доходов и расходов пользователей;
2) категоризация финансовых операций с возможностью создания и редактирования пользовательских категорий;
3) формирование персонального бюджета с установкой лимитов расходов по категориям;
4) контроль превышения заданных лимитов и уведомление пользователя;
5) генерация аналитических отчетов по доходам и расходам за выбранные периоды времени;
6) визуализация финансовых данных с использованием графиков и диаграмм;
7) обеспечение многопользовательского режима с авторизацией и аутентификацией;
8) визуализация финансовых данных с использованием графиков и диаграмм;
9) хранение и обработка истории финансовых операций.

1.2.3 Входные данные

В качестве входных данных для программного средства выступают:

1) данные о доходах пользователя (сумма, источник дохода, дата получения);

2) данные о расходах (сумма, категория, дата, описание операции);

3) пользовательские категории финансовых операций;

4) параметры бюджета (лимиты расходов, временные периоды);

5) регистрационные и авторизационные данные пользователей системы (логин, пароль, электронная почта).

1.2.4 Выходные данные

Выходными данными являются:

1) структурированная информация в виде таблиц и интерфейсных элементов, отображающая текущий баланс, историю операций и распределение расходов;

2) графическая информация, включающая линейные графики, круговые диаграммы и гистограммы, отражающие структуру и динамику финансовых операций;

3) аналитические отчеты по доходам и расходам за заданные периоды времени;

4) уведомления о превышении установленных лимитов бюджета;

5) агрегированные показатели (общий доход, общий расход, остаток средств).

1.2.5 Требования к параметрам технических и программных средств

Для клиентского устройства пользователя аппаратные требования ограничиваются характеристиками, необходимыми для стабильной работы операционной системы и веб-браузера:

1) объем оперативной памяти не менее 4 гигабайт;

2) наличие стабильного интернет-соединения;

3) операционная система Windows, macOS, Linux, iOS или Android.

Для серверной части и разработки программного средства необходимо использование следующих технологий и инструментов:

1) платформа ASP.NET Core для реализации серверной логики приложения;

2) JavaScript-библиотека React для разработки клиентского интерфейса;

3) система управления базами данных (например, PostgreSQL или Microsoft SQL Server);

4) среда разработки Visual Studio или Visual Studio Code;

5) среда выполнения Node.js для поддержки клиентской части.

1.2.6 Требования к информационной и программной совместимост

Доступ к клиентской части системы должен осуществляться через любой современный веб-браузер без необходимости установки

дополнительного программного обеспечения. Интерфейс должен разрабатываться с учетом принципов адаптивного дизайна (Responsive Web Design) для корректного отображения на устройствах с различными разрешениями экранов.

Взаимодействие клиентской и серверной частей приложения должно осуществляться по протоколу HTTP/HTTPS с использованием архитектурного стиля REST. Обмен данными между компонентами системы должен производиться в формате JSON.

Программное средство должно обеспечивать корректную работу на различных операционных системах и устройствах, включая персональные компьютеры, планшеты и мобильные устройства. Хранение и обработка пользовательских данных должны осуществляться с соблюдением требований безопасности, включая использование механизмов аутентификации, авторизации, а также шифрования и хеширования конфиденциальной информации.

1.2.7 Обоснование выбора языка и среды разработки

Выбор технологического стека для разработки программного средства обусловлен требованиями к производительности, надежности, безопасности и распространенности используемых технологий.

В качестве серверной технологии выбран фреймворк ASP.NET Core с использованием языка программирования C#. Данный выбор обусловлен высокой производительностью платформы, что подтверждается результатами независимых сравнительных исследований веб-фреймворков, в которых ASP.NET Core демонстрирует одни из лучших показателей по скорости обработки запросов [9]. Кроме того, язык C# стабильно занимает высокие позиции в рейтингах популярности языков программирования, таких как TIOBE Index и Stack Overflow Developer Survey, что свидетельствует о его востребованности и активной поддержке со стороны сообщества разработчиков [10]. Использование ASP.NET Core также обеспечивает кроссплатформенность и удобство разработки масштабируемых веб-приложений.

Для реализации механизмов аутентификации и авторизации пользователей выбрана система Keycloak. Данное решение широко применяется в корпоративных информационных системах и поддерживает современные стандарты безопасности, такие как OAuth 2.0 и OpenID Connect. Согласно аналитическим обзорам систем управления идентификацией и доступом (IAM), Keycloak является одним из наиболее популярных open-source решений благодаря своей гибкости, расширяемости и высокому уровню безопасности [12].

В качестве системы управления базами данных выбрана PostgreSQL. Данная СУБД занимает лидирующие позиции в рейтингах популярности и надежности баз данных, включая ежегодный опрос Stack Overflow и DB-Engines Ranking [13]. PostgreSQL обеспечивает высокую производительность, поддержку сложных запросов и строгую целостность данных, что делает её подходящей для хранения финансовой информации пользователей.

Для разработки программного средства используется среда JetBrains Rider. Выбор данной среды обусловлен её высокой производительностью и удобством работы с платформой .NET. Согласно исследованиям JetBrains и отзывам разработчиков, Rider обеспечивает более эффективное использование системных ресурсов по сравнению с традиционными IDE, а также предоставляет развитые средства анализа кода и отладки [14].

Таким образом, выбранный технологический стек основан на современных, широко распространенных и проверенных технологиях, что обеспечивает высокую производительность, безопасность и удобство разработки программного средства.

2 Анализ требований к программному средству и разработка спецификаций функциональных требований

2.1 Описание функциональности программного средства

Для описания требований, который были сформулированы для разрабатываемого программного средства, используются UML диаграммы, а конкретно диаграмма вариантов использования. Диаграмма вариантов использования в UML — диаграмма, отражающая отношения между актёрами и прецедентами и являющаяся составной частью модели прецедентов, позволяющей описать систему на концептуальном уровне.

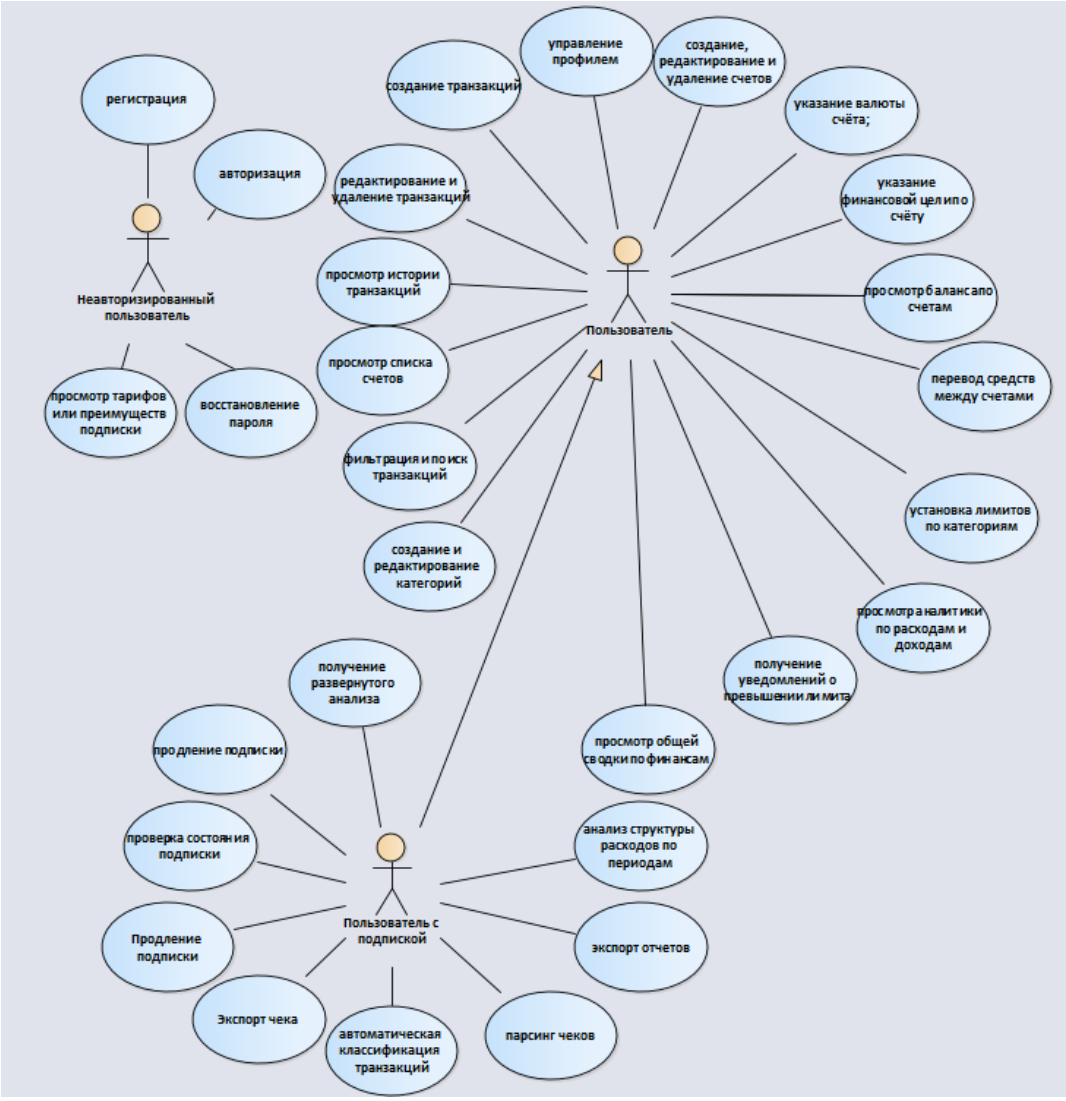

Рисунок 2.1 – Диаграмма вариантов использования

На основании представленной диаграммы вариантов использования можно сделать вывод, что в системе будет существовать три вида пользователей:

– неавторизованный пользователь;

– пользователь;

– пользователь с премиум подпиской.

Основным функционалом обладает пользователь, имеющий доступ к следующим возможностям:

управление учетной записью пользователя;

– создание, редактирование и удаление счетов;

– добавление, изменение и удаление финансовых операций (доходов и расходов);

– управление категориями финансовых операций;

– просмотр истории транзакций и текущего состояния счетов;

– получение уведомлений о финансовых операциях и событиях в системе

Пользователь с премиум-подпиской обладает расширенным набором возможностей. Помимо базового функционала, ему предоставляется доступ к дополнительным инструментам анализа и управления финансами, включая расширенную финансовую аналитику, прогнозирование доходов и расходов, а также использование дополнительных функций системы, направленных на оптимизацию персональных финансов.

2.2 Спецификация функциональных требований

В соответствии с проанализированной предметной областью основной целью разработки является создание программного средства, предназначенного для учёта персональных доходов и расходов, ведения нескольких счетов, анализа финансовых операций, контроля бюджета и оптимизации личных расходов пользователя.

Веб-приложение управления групповыми проектами должно обрабатывать и информировать пользователя об ошибках и успешности операций, быть гибким и независимым от используемой операционной системы. Были сформулированы основные требования к функционалу и пользовательскому интерфейсу:

1) авторизация и регистрация должны включать корректную валидацию вводимых данных, в том числе ограничение длины полей, проверку формата электронной почты и запрет на отправку некорректных данных;

2) в системе должно быть реализовано разграничение доступа к функциям приложения в зависимости от типа пользователя: неавторизованный пользователь, пользователь и пользователь с премиум-подпиской;

3) система должна обеспечивать информативную обработку ошибок и результатов выполнения операций для уведомления пользователя об успешности действий или возникновении ошибок;

4) пользовательский интерфейс должен быть интуитивно понятным, логически организованным и обеспечивать удобный доступ к основным функциям системы;

5) при регистрации у пользователя должны запрашиваться логин, адрес электронной почты и пароль;

6) пользователю должна быть предоставлена возможность изменения личных данных, включая пароль и дополнительные сведения профиля;

7) взаимодействие с приложением должно осуществляться через веб-браузер;

8) система должна обеспечивать возможность ведения нескольких счетов одним пользователем;

9) для каждого счёта должна быть предусмотрена возможность указания валюты и финансовой цели;

10) система должна предоставлять функции ввода, хранения, изменения и анализа финансовых операций

Также детализируются имеющийся требования:

1) авторизация и регистрация:

а) валидация логина, электронной почты и пароля;

б) б) проверка уникальности логина и электронной почты;

в) в) возможность авторизации зарегистрированного пользователя;

г) г) возможность восстановления доступа к учётной записи.

1) управления личным кабинетом:

а) просмотр и редактирование личных данных пользователя;

б) смена пароля;

в) добавление и изменение аватара пользователя;

г) отображение информации о статусе подписки.

2) управление счетами:

а) создание, редактирование и удаление счёта;

б) указание названия счёта;

в) выбор валюты счёта;

г) установка финансовой цели для счёта;

д) просмотр списка счетов пользователя;

е) отображение текущего баланса по каждому счёту;

ж) возможность перевода средств между счетами.

3) управление транзакциями:

а) создание транзакций;

б) разделение транзакций на доходы и расходы;

в) редактирование и удаление транзакций;

г) указание суммы, даты и описания транзакции;

д) привязка транзакции к конкретному счёту;

е) указание категории транзакции;

ж) просмотр истории транзакций.

з) фильтрация и поиск транзакций по дате, категории, типу операции и счёту;

4) управление категориями:

а) создание пользовательских категорий доходов и расходов;

б) редактирование категорий;

в) удаление категорий;

г) использование категорий при добавлении транзакций.

5) управление бюджетом и финансовыми целями:

а) установка лимитов расходов по категориям;

б) контроль превышения установленных лимитов;

в) отображение уведомлений при превышении лимита;

г) отслеживание прогресса достижения финансовой цели по счёту.

6) анализ финансовых данных:

а) отображение общей информации о доходах и расходах;

б) построение аналитических отчётов по финансовым операциям;

в) визуализация структуры расходов и доходов;

г) отображение статистики по категориям, счетам и временным периодам.

7) функции пользователя с премиум-подпиской:

а) использование расширенной аналитики;

б) сравнение финансовых показателей за разные периоды;

в) получение прогноза расходов и доходов;

г) получение рекомендаций по оптимизации расходов;

д) экспорт отчётов в файл;

е) анализ регулярных платежей и подписок;

ж) загрузка изображения чека и распознавание данных;

з) автоматическое создание транзакций на основе распознанного чека;

и) анализ кредитов и кредитной нагрузки пользователя.

2.3 Информационная модель предметной области

Информационная модель предметной области (ИМПО) представляет собой абстрактное и упрощённое описание предметной области управления персональными финансами, предназначенное для последующего проектирования базы данных и программного средства в целом. Данная модель отражает основные сущности системы, их атрибуты и взаимосвязи, фиксируя только те данные, которые необходимы для автоматизации учета доходов, расходов и анализа финансовых операций.

В рамках разрабатываемой информационной системы предметная область включает в себя процессы учета финансовых операций пользователя, управления счетами, категоризации расходов и доходов, а также анализа финансовых данных.

Основными сущностями информационной модели являются:

1) Пользователь — представляет зарегистрированного пользователя системы. Основные атрибуты:

а) идентификатор пользователя;

б) логин;

в) адрес электронной почты;

г) дата регистрации;

д) статус учетной записи.

2) Счёт — отражает финансовые счета пользователя. Основные атрибуты:

а) идентификатор счёта;

б) наименование счёта;

в) текущий баланс;

г) валюта.

3) Финансовая операция — описывает движение денежных средств. Основные атрибуты:

а) идентификатор операции;

б) тип операции (доход/расход);

в) сумма;

г) дата операции;

д) описание.

4) Категория — используется для классификации финансовых операций. Основные атрибуты:

а) идентификатор категории;

б) наименование категории;

в) тип категории (доход или расход).

5) Бюджет — определяет ограничения расходов по категориям за определённый период. Основные атрибуты:

а) идентификатор бюджета;

б) лимит расходов;

в) временной период.

6) Подписка — отражает наличие у пользователя расширенного функционала системы. Основные атрибуты:

а) идентификатор подписки;

б) тип подписки;

в) дата начала;

г) дата окончани.

Между выделенными сущностями существуют следующие взаимосвязи:

– один пользователь может иметь несколько счетов;

– один пользователь может иметь множество финансовых операций;

– каждая финансовая операция связана с одним счётом;

– каждая операция относится к одной категории, при этом одна категория может использоваться во множестве операций;

– пользователь может задавать несколько бюджетов для различных категорий;

– пользователь может иметь одну активную подписку, определяющую доступ к расширенному функционалу системы.

2.4 Нефункциональные требования

Нефункциональные требования определяют качественные характеристики разрабатываемого программного обеспечения: его производительность, эргономику, отказоустойчивость, безопасность и масштабируемость. Поскольку проект реализуется как современное веб-приложение, данные требования в значительной степени опираются на возможности выбранного технологического стека (ASP.NET Core, React, PostgreSQL, Keycloak), обеспечивающего высокую производительность, надежность и безопасность системы.

К программному средству предъявляются следующие нефункциональные требования:

– Время первоначальной загрузки клиентского приложения, а также отклика интерфейса на действия пользователя не должно превышать нескольких секунд. Формирование отчетов и обработка запросов к серверу должны выполняться в минимально возможное время за счет использования высокопроизводительного серверного фреймворка ASP.NET Core и оптимизированных запросов к базе данных PostgreSQL;

– Система должна корректно обрабатывать ошибки, возникающие при взаимодействии клиентской и серверной частей. В случае сетевых сбоев, ошибок сервера или некорректных пользовательских действий должны отображаться информативные сообщения, не приводящие к аварийному завершению работы приложения. Все критические операции должны сопровождаться обработкой исключений;

– Масштабируемость и расширяемость. Архитектура приложения должна обеспечивать возможность дальнейшего расширения функционала без значительной переработки существующего кода. Использование модульной архитектуры и компонентного подхода (React) позволяет добавлять новые элементы интерфейса и функциональные модули. Серверная часть на ASP.NET Core должна поддерживать горизонтальное масштабирование при увеличении нагрузки;

– Доступ к системе должен осуществляться через веб-браузер без необходимости установки дополнительного программного обеспечения. Интерфейс должен корректно отображаться на различных устройствах (персональные компьютеры, планшеты, мобильные устройства) и операционных системах.

В рамках данного раздела было выполнено концептуальное моделирование разрабатываемого программного средства с использованием диаграммы вариантов использования (Use Case), что позволило четко определить границы системы и разграничить прецеденты взаимодействия для

ролей пользователя, администратора и внешних устройств. На основе построенной модели разработана подробная спецификация функциональных требований. Кроме того, сформулированные нефункциональные требования к производительности, информационной безопасности, масштабируемости и эргономике обеспечивают исчерпывающую техническую базу, необходимую для перехода к этапу непосредственного архитектурного проектирования и разработки алгоритмов.

3 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

3.1 Разработка архитектуры программного средства

В данном разделе будет рассмотрено, какой архитектурный подход и технологии были выбраны для проектирования программного средства.

3.1.1 Общая архитектура приложения

За основу разработки программного средства выбрана клиент-серверная архитектура с использованием принципа Single Page Application (SPA) и подхода чистой архитектуры (Clean Architecture), предполагающего разделение системы на независимые слои: Domain, Application, Infrastructure и Presentation.

Клиент-серверная архитектура представляет собой распределённую модель построения программного обеспечения, при которой функции системы разделяются между клиентской и серверной частями. Клиентская часть отвечает за взаимодействие с пользователем, тогда как серверная обеспечивает обработку данных, реализацию бизнес-логики и взаимодействие с базой данных. Обмен данными между клиентом и сервером осуществляется по протоколу HTTP/HTTPS с использованием REST API.

В рамках подхода Single Page Application клиентская часть реализована в виде одностраничного веб-приложения, разработанного с использованием библиотеки React. Данный подход позволяет загружать единственную HTML-страницу, а дальнейшее обновление интерфейса осуществляется динамически без полной перезагрузки страницы. Для взаимодействия с сервером используются асинхронные запросы например, с помощью Fetch API или библиотеки Axios. Маршрутизация осуществляется на стороне клиента, что позволяет изменять отображаемые компоненты в зависимости от URL без перезагрузки страницы.

В основе серверной части лежит концепция чистой архитектуры, направленная на разделение ответственности и снижение связанности компонентов системы. Архитектура включает следующие уровни:

1) Presentation layer (слой представления) – Данный слой обеспечивает взаимодействие системы с внешними пользователями и клиентским приложением. В него входят контроллеры ASP.NET Core, принимающие HTTP-запросы, обрабатывающие входные данные и возвращающие ответы в формате JSON. Также данный слой отвечает за валидацию входных данных и передачу запросов в слой приложения;

2) Application layer (слой приложения) – Слой приложения содержит реализацию бизнес-сценариев системы (use cases). Он координирует выполнение операций, обрабатывает запросы от слоя представления и взаимодействует с доменной моделью. В данном слое реализуются сервисы приложения, обработчики команд и запросов, а также логика управления пользовательскими действиями;

3) Domain layer (доменный слой) – Данный слой содержит основные бизнес-сущности системы и бизнес-правила. Он является независимым от внешних технологий и не зависит от других слоёв. В доменном слое описываются ключевые сущности предметной области, такие как пользователь, счёт, финансовая операция, категория и бюджет, а также их поведение;

4) Infrastructure layer (инфраструктурный слой) – Инфраструктурный слой обеспечивает взаимодействие системы с внешними ресурсами, такими как база данных PostgreSQL, сервисы аутентификации (Keycloak) и другие внешние компоненты. В данном слое реализуются репозитории, механизмы доступа к данным, интеграции и технические детали реализации.

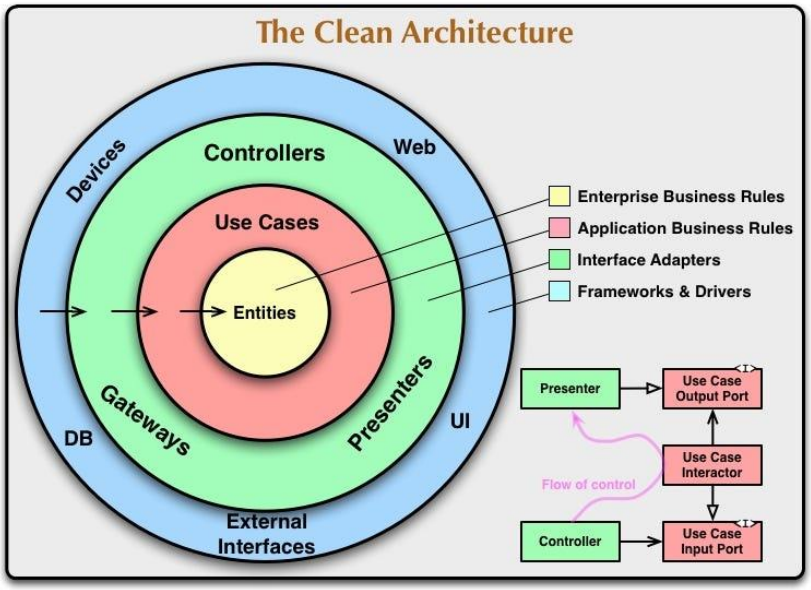

Рисунок 3.1 – Общая структура приложения с чистой архитектурой

3.1.2 Архитектура клиентской части

В качестве основы для разработки клиентской части программного средства планируется использование библиотеки React, предназначенной для создания пользовательских интерфейсов. React является одной из наиболее распространённых технологий для разработки веб-приложений и позволяет реализовывать динамичные и отзывчивые интерфейсы.

Применение React обусловлено его компонентным подходом, при котором пользовательский интерфейс формируется из отдельных переиспользуемых элементов (компонентов). Такой подход упрощает разработку, повышает читаемость кода и облегчает дальнейшее расширение функциональности приложения. Каждый компонент может обладать собственным состоянием и логикой, что обеспечивает гибкость при разработке сложных интерфейсов.

Клиентская часть будет реализована в формате одностраничного приложения (Single Page Application), при котором загрузка основной страницы происходит один раз, а последующее обновление содержимого осуществляется динамически без полной перезагрузки страницы. Это позволяет повысить производительность интерфейса и улучшить пользовательский опыт.

Для организации навигации внутри приложения предполагается использование механизмов клиентской маршрутизации, позволяющих изменять отображаемое содержимое в зависимости от текущего адреса страницы без повторной загрузки данных с сервера.

Управление состоянием приложения может осуществляться с использованием встроенных возможностей React, а также при необходимости с применением дополнительных инструментов для централизованного хранения и обработки данных. Такой подход позволяет эффективно организовать передачу данных между компонентами и обеспечить согласованность состояния интерфейса.

Также при разработке могут использоваться современные средства работы с асинхронными запросами, обеспечивающие взаимодействие клиентской части с сервером, включая получение и отправку данных, а также обработку результатов выполнения операций.

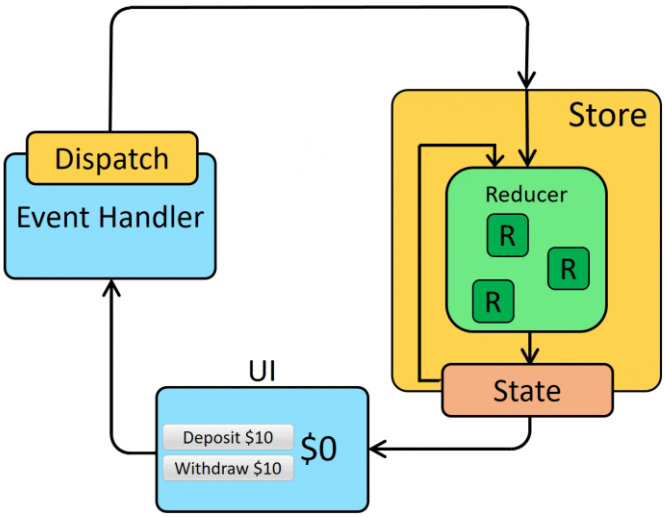

Рисунок 3.2 – Схема управления состоянием в клиентском приложении

В процессе разработки клиентской части важную роль играет организация управления состоянием приложения, которое отражает текущие данные и их изменения в пользовательском интерфейсе.

В современных веб-приложениях управление состоянием может осуществляться как с использованием встроенных механизмов библиотеки React, так и с применением специализированных инструментов централизованного хранения данных. Выбор конкретного подхода зависит от сложности приложения, объёма данных и требований к масштабируемости.

При разработке системы предполагается использование централизованного подхода к управлению состоянием, при котором данные хранятся в едином логическом хранилище и передаются между компонентами по установленным правилам. Такой подход позволяет обеспечить согласованность данных, упростить их обновление и повысить предсказуемость поведения интерфейса.

Обновление состояния осуществляется через определённые механизмы обработки событий, при которых формируется новое состояние на основе предыдущего, что исключает прямое изменение данных и повышает надёжность работы приложения.

Для выполнения асинхронных операций, таких как взаимодействие с сервером и получение данных, могут использоваться дополнительные механизмы обработки запросов, позволяющие корректно обрабатывать результаты выполнения операций и обновлять состояние приложения.

3.1.3 Архитектура серверной части

В качестве основы для серверной части разрабатываемого программного средства используется платформа ASP.NET Core, представляющая собой

современный кроссплатформенный фреймворк для создания веб-приложений и сервисов. Данная технология разработана компанией Microsoft и широко применяется при создании высоконагруженных и масштабируемых систем.

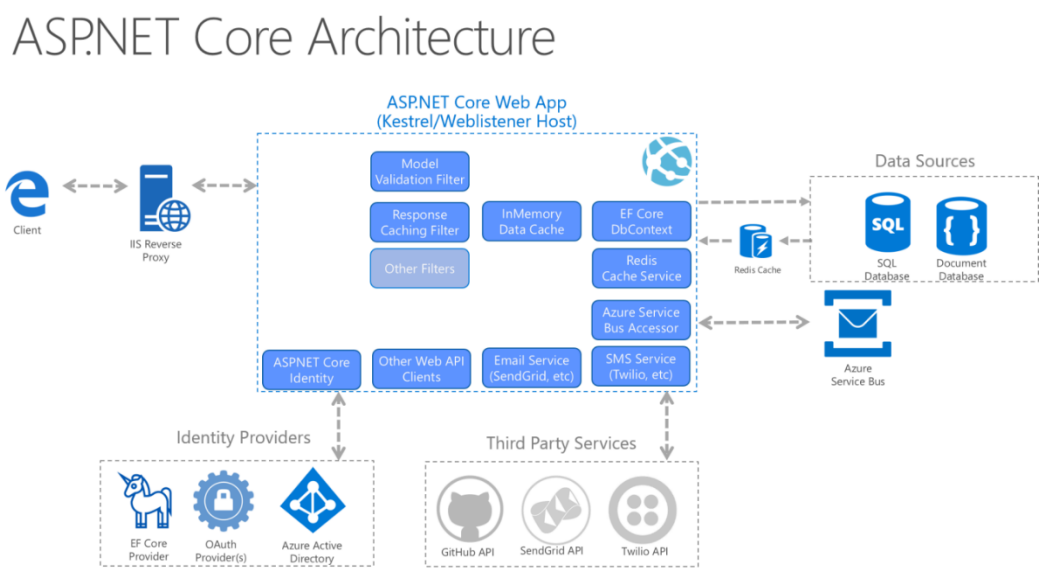

Рисунок 3.3 – Структура работы ASP.NET

ASP.NET Core обеспечивает высокую производительность обработки HTTP-запросов, встроенную поддержку разработки REST API, а также гибкую архитектуру, позволяющую реализовывать различные подходы к построению программного обеспечения, включая чистую архитектуру (Clean Architecture). Серверная часть приложения реализуется на языке программирования C#, который обладает строгой типизацией, что способствует повышению надежности и сопровождаемости кода.

Взаимодействие клиентской и серверной частей осуществляется посредством HTTP/HTTPS-запросов. Сервер обрабатывает входящие запросы, выполняет необходимую бизнес-логику, обращается к базе данных и возвращает результат. Такой подход обеспечивает независимость клиентской части и позволяет использовать различные типы клиентов (веб, мобильные приложения и др.).

Для обработки запросов в ASP.NET Core используются контроллеры, которые принимают входные данные, выполняют их валидацию и передают управление в слой приложения. В рамках архитектуры программного средства серверная часть разделена на логические уровни, что обеспечивает разделение ответственности и упрощает сопровождение системы.

Особое внимание в разрабатываемом программном средстве уделяется вопросам информационной безопасности. Для реализации механизмов аутентификации и авторизации пользователей используется система

управления доступом Keycloak. Данное решение позволяет централизованно управлять учетными записями пользователей и поддерживает современные протоколы безопасности, такие как OAuth 2.0 и OpenID Connect.

Использование Keycloak позволяет реализовать безопасную аутентификацию пользователей без необходимости разработки собственной системы управления учетными записями. При входе в систему пользователь перенаправляется на сервер аутентификации, где происходит проверка учетных данных. После успешной аутентификации пользователю выдается токен доступа, который используется для последующих запросов к серверной части приложения.

Сервер проверяет полученный токен и, в зависимости от роли пользователя, предоставляет доступ к соответствующим функциям системы. Такой подход позволяет обеспечить разграничение прав доступа и повысить уровень защищенности программного средства.

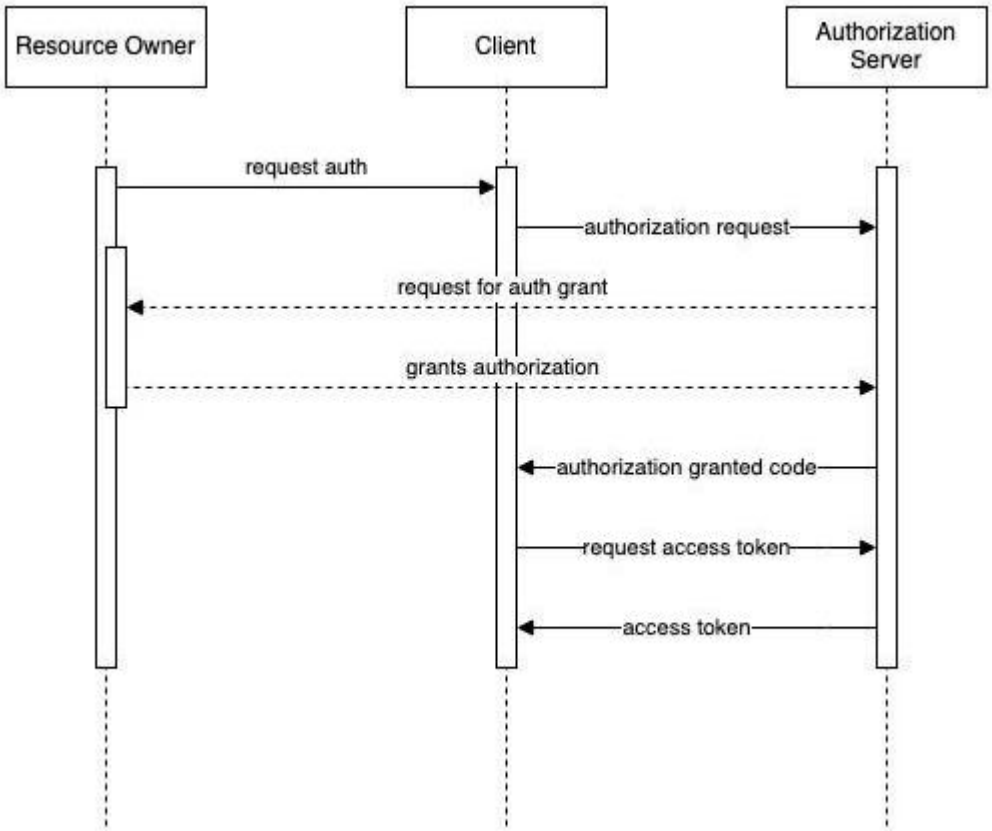

Рисунок 3.4 – Структура авторизации в Keycloak

3.1.4 База данных

В качестве системы управления базами данных для разрабатываемого программного средства выбрана PostgreSQL — современная объектно-

реляционная СУБД с открытым исходным кодом, широко применяемая при создании веб-приложений и корпоративных информационных систем.

В отличие от встраиваемых решений, PostgreSQL реализует классическую клиент-серверную архитектуру, при которой база данных функционирует как отдельный сервер, обеспечивающий централизованное хранение, обработку и защиту данных. Такой подход позволяет эффективно работать с многопользовательскими системами, обеспечивая высокую степень надежности и масштабируемости.

PostgreSQL поддерживает широкий спектр возможностей, включая транзакции, сложные запросы, индексацию, механизмы обеспечения целостности данных, а также расширяемость за счет использования пользовательских типов и функций. Это делает данную СУБД особенно подходящей для хранения и обработки финансовой информации, требующей высокой точности и надежности.

К основным преимуществам PostgreSQL можно отнести:

– высокую производительность при работе с большими объемами данных;

– развитые механизмы обеспечения целостности данных;

– возможность масштабирования системы;

– активное развитие и поддержку со стороны сообщества;

– кроссплатформенность и бесплатное распространение.

Для взаимодействия серверной части приложения с базой данных используется технология Entity Framework Core — объектно-реляционное отображение (ORM), позволяющее работать с данными на уровне объектов языка программирования C#.

Entity Framework Core обеспечивает абстракцию над базой данных, позволяя разработчику выполнять операции создания, чтения, обновления и удаления данных без необходимости написания SQL-запросов вручную. Вместо этого используются классы и объекты, соответствующие сущностям предметной области.

К основным преимуществам PostgreSQL можно отнести:

– ускорить процесс разработки за счет автоматизации работы с базой данных;

– повысить читаемость и поддерживаемость кода;

– минимизировать количество ошибок при работе с данными;

– обеспечить независимость бизнес-логики от конкретной СУБД;

– использовать механизм миграций для управления структурой базы данных.

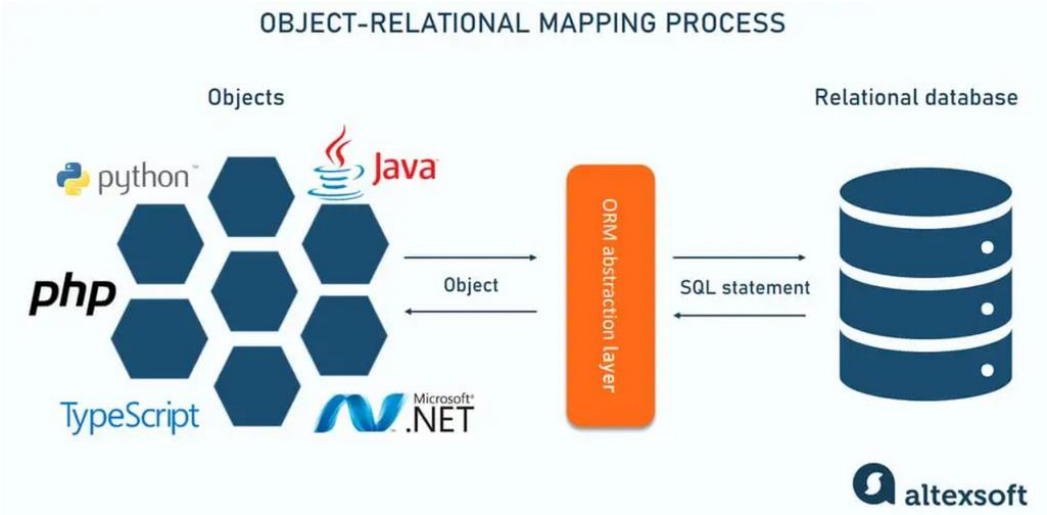

Рисунок 3.5 – Структура работы Entity framework

3.2 Разработка физической модели базы данных

На данном этапе была разработана физическая модель базы данных. Физическая модель базы данных учитывает типы данных, размеры полей, связи между таблицами и ограничения целостности данных. Физическая модель базы данных представлена на рисунке 3.7.

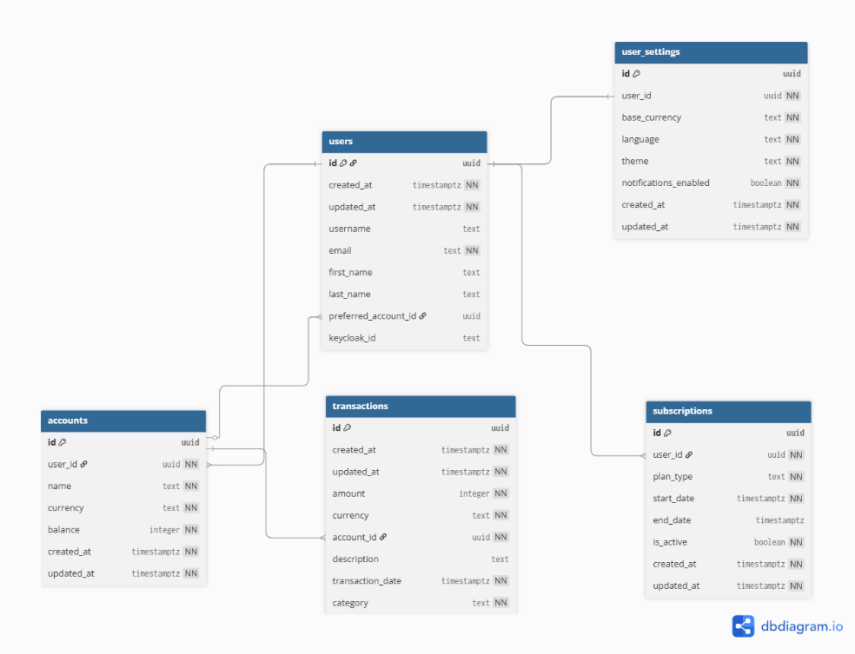

Рисунок 3.6 – Физическая модель базы данных

В физической модели базы данных представлены сущности, описанные в таблице 3.1.

Таблица 3.1 – Сущности базы данных

Сущность	Описание сущности	Поля	Описание полей
1	2	3	4
Users	Хранит данные пользователей системы	Id: uuid CreatedAt: timestamp with time zone UpdatedAt: timestamp with time zone Username: text Email: text FirstName: text LastName: text PreferredAccountId: uuid KeycloakId: text	Идентификатор пользователя Дата создания записи Дата обновления записи Имя пользователя Почта Имя Фамилия Идентификатор основного счета Идентификатор пользователя в Keycloak
Accounts	Хранит данные о счетах пользователей	Id: uuid UserId: uuid Name: text Currency: text Balance: integer CreatedAt: timestamp with time zone UpdatedAt: timestamp with time zone	Идентификатор счета Идентификатор пользователя Имя счета Валюта счета Текущий баланс Дата создания записи Дата обновления записи

Продолжение таблицы 3.1

1	2	3	4
Transactions	Хранит данные о финансовых операциях пользователей	Id: uuid CreatedAt: timestamp with time zone UpdatedAt: timestamp with time zone Amount: integer Currency: text AccountId: uuid Description: text TransactionDate: timestamp with time zone Category: text	Идентификатор операции Дата создания записи Дата обновления записи Сумма операции Валюта операции Идентификатор счета Описание Дата совершения операции Категория операции
Subscriptions	Хранит данные о подписках пользователей	Id: uuid UserId: uuid PlanType: text StartDate: timestamp with time zone EndDate: timestamp with time zone IsActive: boolean CreatedAt: timestamp with time zone UpdatedAt: timestamp with time zone	Идентификатор подписки Идентификатор пользователя Тип подписки Дата начала подписки Дата окончания подписки Признак активности Дата создания записи Дата обновления записи

Продолжение таблицы 3.1

1	2	3	4
UserSettings	Хранит пользовательские настройки системы	Id: uuid UserId: uuid BaseCurrency: text Language: text Theme: text CreatedAt: timestamp with time zone UpdatedAt: timestamp with time zone	Идентификатор настроек Идентификатор пользователя Основная валюта Язык интерфейса Тема оформления Дата создания записи Дата обновления записи

3.3 Разработка алгоритма ПС и алгоритмов отдельных модулей

На данном этапе начинается процесс разработки алгоритма программного обеспечения и алгоритмов отдельных вспомогательных модулей, обеспечивающих выполнение функциональных требований к приложению. В данном подразделе, представляются описания и схемы алгоритмов, последние согласно ГОСТ 19.701-90.

3.3.1 Обобщённый алгоритм программного средства

Обобщённый алгоритм работы программного средства описывает основные этапы взаимодействия пользователя с системой анализа и оптимизации персональных расходов.

При запуске приложения выполняется проверка наличия JWT-токена. Если пользователь ранее был авторизован, выполняется автоматический вход в систему. В противном случае пользователю предлагается пройти авторизацию или регистрацию. При регистрации вводятся логин, электронная почта и пароль, которые проходят валидацию.

После успешной авторизации пользователь получает доступ к функционалу системы, включая добавление, редактирование и удаление финансовых операций, их категоризацию, а также формирование бюджета и контроль расходов. Система предоставляет инструменты анализа данных, такие как отчёты и визуализация расходов

Пользователь может завершить работу с системой, выполнив выход из аккаунта.

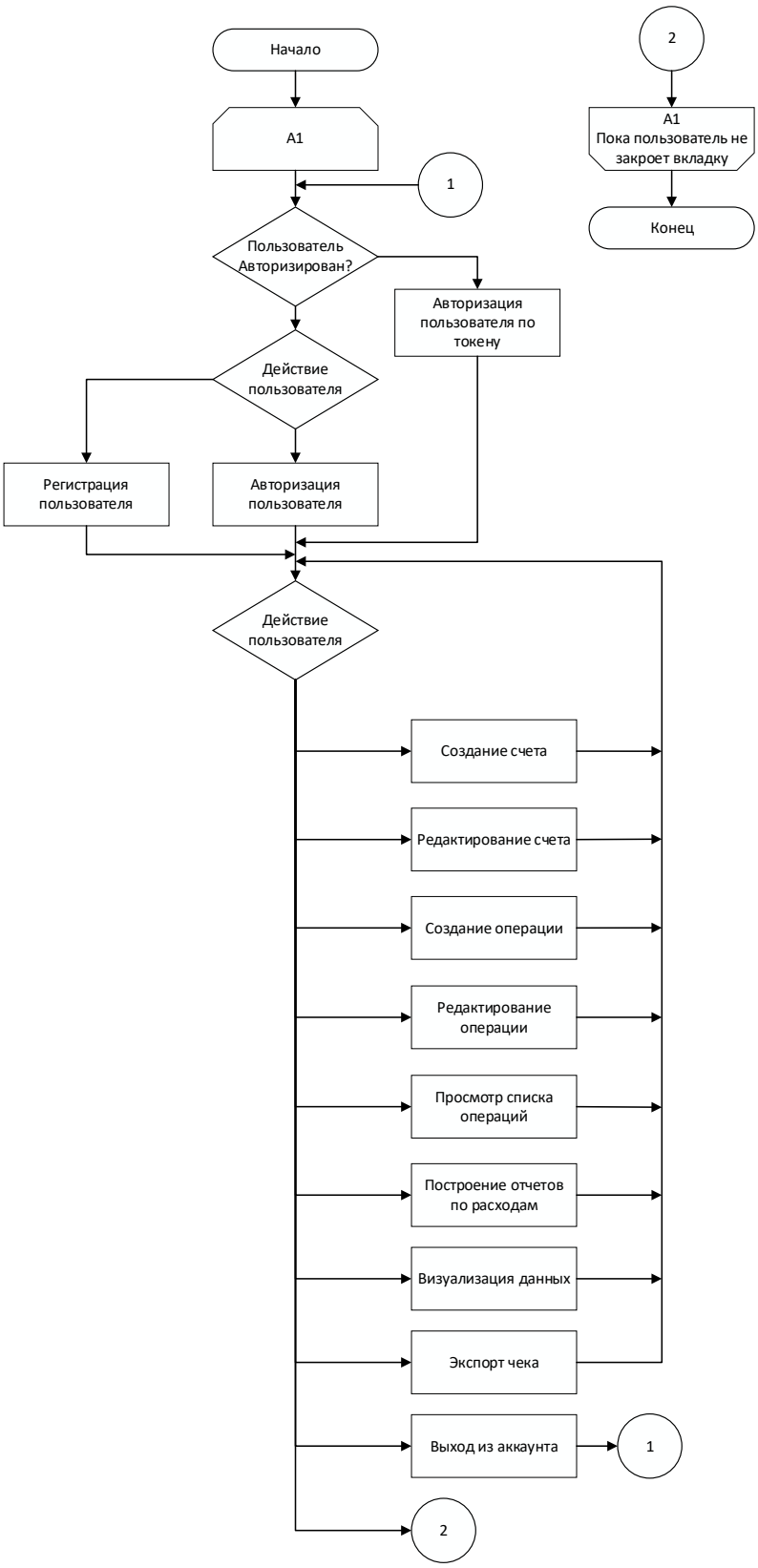

Рисунок 3.7 – Схема программы

Пользователь может создавать и редактировать счета, предназначенные для хранения информации о денежных средствах.

Пользователь имеет возможность создавать и редактировать финансовые операции, а также просматривать список всех операций с возможностью их анализа.

Система предоставляет функции построения отчетов по расходам и визуализации данных в виде графиков и диаграмм для удобного восприятия информации.

Пользователь может выполнить экспорт чека по выбранной операции для сохранения или дальнейшего использования.

Также пользователь может завершить работу с системой, выполнив выход из аккаунта.

Схема программы представлена на рисунке 3.7.

3.3.2 Алгоритм взаимодействия с уведомлениями

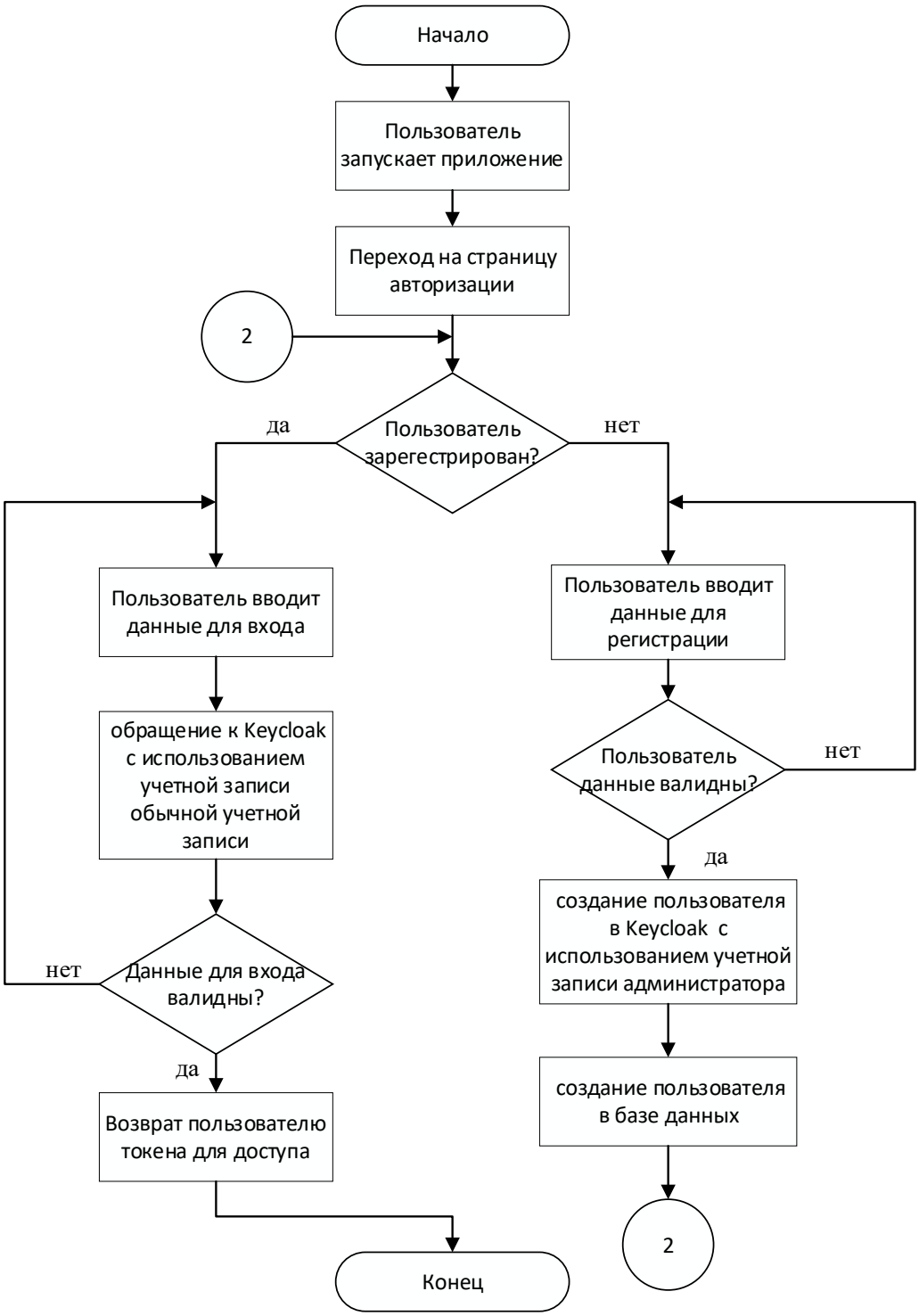

Рисунок 3.8 – Схема подпроцесса взаимодействия с уведомлениями

На рисунке 3.8 представлена схема подпроцесса аутентификации и регистрации пользователя с использованием системы управления доступом Keycloak.

При регистрации пользователя клиентское приложение передаёт данные (логин, электронная почта, пароль) на сервер. Сервер, в свою очередь,

взаимодействует с системой Keycloak через административный API с использованием учётных данных администратора. В Keycloak создаётся новый пользователь с указанными параметрами.

При выполнении авторизации пользователь вводит свои учетные данные, которые передаются на сервер. Сервер инициирует запрос к Keycloak от имени пользователя и получает JWT-токен в случае успешной аутентификации. Полученный токен возвращается клиенту и используется для последующих запросов к защищённым ресурсам системы.

При последующих обращениях клиента к серверу выполняется проверка валидности JWT-токена. В случае успешной проверки пользователь получает доступ к функционалу системы. При отсутствии или истечении срока действия токена требуется повторная авторизация.

Данный подход обеспечивает централизованное управление пользователями, безопасность хранения учетных данных и возможность масштабирования системы.

3.3.3 Алгоритм полного взаимодействия с карточками

На рисунке 3.19 представлена схема подпроцесса формирования аналитических отчетов по расходам на стороне клиента и сервера. Пользователь выбирает период анализа и при необходимости задаёт фильтры, после чего отправляется запрос на сервер.

Сервер получает данные из базы данных. При их отсутствии возвращается сообщение об ошибке. В случае наличия данных выполняется их обработка: для каждой операции определяется категория и временной интервал, проверяется наличие категории в отчете и добавляется сумма операции в соответствующую группу. Обработка выполняется циклически до завершения всех операций.

После этого рассчитываются итоговые значения доходов и расходов, формируются агрегированные данные и передаются на клиент, где отображаются в виде графиков и диаграмм.

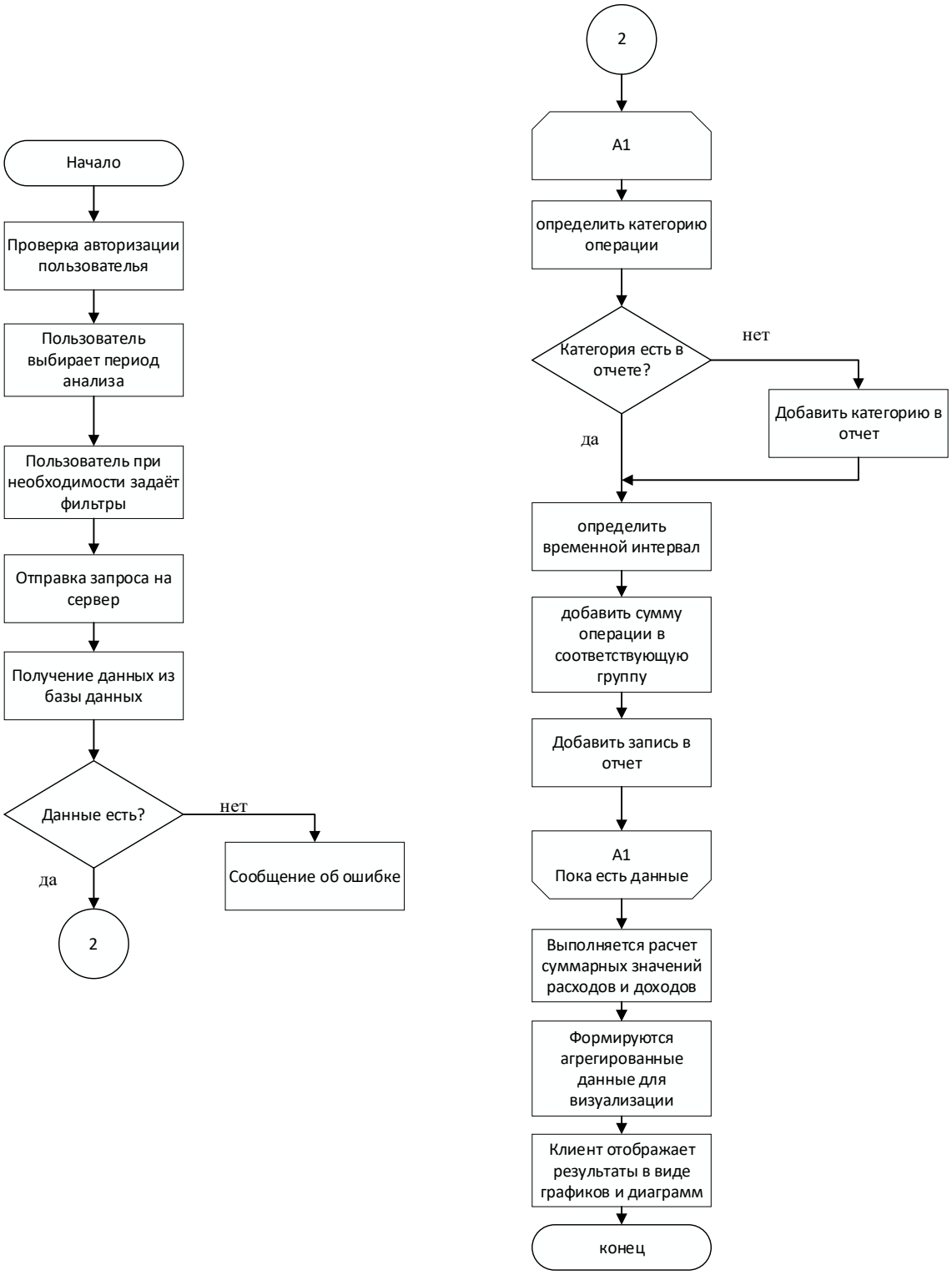

Рисунок 3.9 – Схема подпроцесса полного взаимодействия с карточками

3.3.4 Общий принцип алгоритмов

Большинство алгоритмов, реализованных в разрабатываемом веб-приложении, имеют схожую структуру и соответствуют паттерну CRUD (Create, Read, Update, Delete). Основные сущности системы, особенно на

серверной стороне, проходят стандартные этапы жизненного цикла, включающие создание, чтение, обновление и удаление данных.

Алгоритмы клиентской части приложения ориентированы на обработку пользовательских действий, отображение данных, а также передачу запросов на сервер. Кроме того, реализуются механизмы взаимодействия с локальным состоянием приложения и обмена данными между компонентами.

В связи с высокой степенью повторяемости и унификации данных процессов для различных сущностей системы, соответствующие алгоритмы имеют типовую структуру и реализуются по единым принципам.

ЗАКЛЮЧЕНИЕ

В рамках преддипломной практики были получены и обоснованы основные результаты, необходимые для дальнейшей разработки дипломного проекта, направленного на повышение эффективности управления личными финансами пользователя за счёт создания системы анализа и оптимизации персональных расходов.

В ходе выполнения практики был проведён анализ предметной области, информационных источников и существующих программных решений. Это позволило выявить их основные особенности, преимущества и недостатки, а также определить направления совершенствования разрабатываемой системы.

На основе проведённого анализа было разработано задание на дипломное проектирование и сформировано техническое задание на разработку программного средства. Кроме того, выполнен анализ требований к системе и определены функциональные требования, обеспечивающие реализацию основных возможностей по учёту и анализу персональных расходов.

В рамках практики также было выполнено проектирование архитектуры программного средства, определены его основные компоненты и их взаимодействие. Разработан обобщённый алгоритм функционирования системы, а также алгоритмы отдельных модулей, обеспечивающих обработку и анализ финансовых данных.

Результатом преддипломной практики является подготовленная основа для дипломного проекта, представленная в виде первых трёх разделов пояснительной записки. В первом разделе выполнен анализ предметной области и сформированы требования к системе. Во втором разделе разработана архитектура программного средства. В третьем разделе описаны основные аспекты реализации и алгоритмического обеспечения системы.

Таким образом, цель преддипломной практики достигнута, а полученные результаты обеспечивают необходимую базу для последующей реализации и завершения дипломного проекта.

СПИСОК ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ

[1] Personal Finance Apps Market Size, Share & Trends Analysis Report [Электронный ресурс]. – Электронные данные – Режим доступа : https://www.zionmarketresearch.com/report/personal-finance-apps-market

[2] Personal Finance App Industry Statistics 2025 // CoinLaw [Электронный ресурс]. – Электронные данные – Режим доступа : https://coinlaw.io/personal-finance-app-industry-statistics/

[3] Personal Finance Mobile App Market Research Report [Электронный ресурс]. – Электронные данные – Режим доступа : https://www.marketresearchfuture.com/reports/personal-finance-mobile-app-market-22409

[4] Davenport T., Harris J. *Competing on Analytics* [Электронный ресурс]. – Электронные данные – Режим доступа : https://hbr.org/product/competing-on-analytics-updated-with-a-new-introduction/10257-PBK-ENG

[5] Han J., Kamber M., Pei J. *Data Mining: Concepts and Techniques* [Электронный ресурс]. – Электронные данные – Режим доступа : https://www.sciencedirect.com/book/9780123814791/data-mining-concepts-and-techniques

[6] Nielsen J. *Usability Engineering* [Электронный ресурс]. – Электронные данные – Режим доступа : https://www.nngroup.com/books/usability-engineering/

[7] Stallings W. *Cryptography and Network Security* [Электронный ресурс]. – Электронный данные – Режим доступа : https://www.pearson.com/en-us/subject-catalog/p/cryptography-and-network-security-principles-and-practice

[8] OECD. Digitalisation and Finance Report [Электонный ресурс]. – Электронные данные – Режим доступа : https://www.oecd.org/

[9] TechEmpower. Web Framework Benchmarks [Электонный ресурс]. – Электронные данные – Режим доступа : https://www.techempower.com/benchmarks/

[10] Stack Overflow Developer Survey [Электонный ресурс]. – Электронные данные – Режим доступа : https://survey.stackoverflow.co/

[11] TIOBE Index [Электонный ресурс]. – Электронные данные – Режим доступа : https://www. tiobe.com/tiobe-index/

[12] Red Hat. Keycloak Overview [Электронный ресурс]. – Электронные данные – Режим доступа : https://www.keycloak.org/

[13] DB-Engines Ranking [Электронный ресурс]. – Электронные данные – Режим доступа : https://db-engines.com/en/ranking

[14] JetBrains. Developer Ecosystem Survey 2023 [Электронный ресурс]. – Электронные данные – Режим доступа : https://www.jetbrains.com/lp/devecosystem-2023/